Chapter 2 · Lesson 2 — Scaffolding the Project

Chapter 2 · Lesson 2 — Scaffolding the Project

> *Goal: from an empty folder to a running Next.js dev server, with every config file explained line by line. By the end of this lesson, `npm run dev` boots a placeholder page at `http://localhost:3000`.*

The two ways to start a Next.js project

There are two common starting points:

1. **`npx create-next-app@latest`** — interactive scaffold. Asks you a dozen questions and generates a full project. Recommended for **most** real projects.
2. **Build the files by hand.** Manual scaffold. Slower but you understand every line. Recommended for **learning**.

This lesson does Option 2. You'll scaffold by hand. By the end you'll know what `create-next-app` would have done for you, and why.

Step 0: Create the folder

```
# In PowerShell on Windows
New-Item -ItemType Directory -Path "snooker-fantasy" | Out-Null
Set-Location "snooker-fantasy"
```

(If you're following along inside the existing repo, you can skip this — just imagine the rest of the lesson playing out in an empty folder.)

Step 1: `package.json`

This is where every JavaScript project starts. Create a file called `package.json` in the project root with this content:

```
{
  "name": "snooker-fantasy",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "format": "prettier --write \"**/*.{ts,tsx,md,json}\""
  },
  "dependencies": {
    "next": "14.2.15",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "lucide-react": "^0.453.0",
    "recharts": "^2.13.0"
  },
  "devDependencies": {
    "typescript": "^5.6.2",
    "@types/react": "^18.3.11",
    "@types/react-dom": "^18.3.0",
    "@types/node": "^20.16.10",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.13",
    "eslint": "^8.57.1",
    "eslint-config-next": "14.2.15",
    "prettier": "^3.3.3"
  }
}
```

Walk through every key:

`"name"`, `"version"`, `"private"`

- **`"name"`** is the project name. Used by tools, never by your code.
- **`"version"`** follows semver (semantic versioning). Doesn't matter for an unpublished app; matters a lot for npm packages.
- **`"private": true"`** is a safety latch. It tells npm *"do not publish this to the public registry."* Don't omit it. Many beginners have accidentally published their unfinished apps because they didn't have this.

`"scripts"` — the npm command surface

These are the commands you'll type 100 times during development:

- **`npm run dev`** → `next dev` — starts the development server with hot reloading at `http://localhost:3000`.
- **`npm run build`** → `next build` — produces an optimized production build in `.next/`. This is what runs on every CI / Vercel deploy.
- **`npm start`** → `next start` — runs the production build (after `build`).
- **`npm run lint`** → `next lint` — runs ESLint with Next's preset rules.
- **`npm run format`** → `prettier --write "**/*.{ts,tsx,md,json}"` — auto-formats every file.

You'll mostly live in `dev` and occasionally `build`. The others are CI / deploy concerns.

> **Why does `npm run dev` need `run` but `npm start` doesn't?** Historical: a few script names (`start`, `test`, `install`, `restart`, `stop`) are built-in shortcuts; the rest need `run`. It's a wart. Live with it.

`"dependencies"` — what ships in production

Five packages:

- **`next`** — the framework itself. Pinned to a specific version (`14.2.15`) intentionally; we want reproducible builds. The ^ in front of others means "any compatible minor version."
- **`react`** and **`react-dom`** — React itself plus the DOM renderer. They're separate so React Native or other renderers can swap `react-dom` out.

- **`lucide-react`** — a beautiful, tree-shakeable icon library. We use it for the tab icons (Trophy, Calendar, Target, Users, BarChart3) and a handful of other places.
- **`recharts`** — declarative charts (covered in Chapter 6).

That's it. Notice what's **not** here: no Redux, no axios, no moment, no lodash, no react-router-dom (Next.js has its own routing). **Senior projects have small dependency lists.** Every package you add is a maintenance debt.

`"devDependencies"` — what's needed only at dev/build time

These are tooling: TypeScript itself, type definitions for React/Node, Tailwind/PostCSS, ESLint, Prettier. They don't ship in the production bundle.

The split between `dependencies` and `devDependencies` is **discipline**. If you accidentally put `prettier` in `dependencies`, your production bundle will include Prettier (10MB) for no reason. Always put dev-only tools in `devDependencies`.

Step 2: `tsconfig.json` (TypeScript compiler config)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./*"] }
  },
```

```
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules", "_source"]
}
```

The most important keys, in plain English:

- **`"target": "ES2022"`** — compile down to JavaScript syntax that's at most "ES2022" (modern, well-supported). Used to be "ES5" for IE11 days; that's history.
- **`"strict": true`** — turn on all strict checks: no implicit any, strict null checks, etc. **Don't disable this.** It's the whole point of TypeScript.
- **`"noEmit": true`** — TypeScript only checks types; it doesn't output `.js` files. Next.js handles compilation via SWC (a Rust-based compiler).
- **`"jsx": "preserve"`** — leave JSX in place; let Next.js compile it. (Other values like `"react"` would compile JSX to `React.createElement` calls in TypeScript itself; Next.js prefers to do that step.)
- **`"paths": { "@/*": ["./*"] }`** — the **path alias**. Lets you write `import X from '@/lib/types'` instead of `'../../../lib/types'`. The `@/` is a convention; you could pick any prefix (`~/`, `$/`, etc.). Most Next.js projects use `@/`.
- **`"plugins": [{ "name": "next" }]`** — enables Next.js-specific TypeScript features (e.g. typed routes).
- **`"include"`** lists which files TypeScript should check; **`"exclude"`** explicitly skips `_source/` (where the original .jsx file lives) and `node_modules`.

If you don't fully understand every flag, that's fine. **Most TS configs in the wild are copy-paste from a working project**, including in senior repos. The key flags to actually understand are `strict`, `paths`, and `noEmit`. The rest is plumbing.

Step 3: `next.config.mjs`

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
};
export default nextConfig;
```

That's the entire config. Two notes:

- The **`/** @type ... */`** comment is JSDoc — it tells your editor (TypeScript or VS

Code) what shape this object should have, even though the file is `.mjs` (no TS). You get autocomplete on `nextConfig` properties for free.

- **`reactStrictMode: true`** double-invokes some React lifecycle calls during development to surface bugs (e.g. side effects in render, deprecated APIs). It's free; leave it on.

`.mjs` vs `.js`? Next.js wants config files in ESM (`import`/`export`) format. The `.mjs` extension forces Node to treat the file as ESM regardless of `"type": "module"` in `package.json`. Belt and braces.

Step 4: `tailwind.config.ts`

```ts
import type { Config } from 'tailwindcss';

const config: Config = {
  content: [
    './app/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
  ],
  theme: { extend: {} },
  plugins: [],
};
export default config;
```

The only key that matters here is **`content`**. It tells Tailwind where to scan for class names so it can generate only the CSS you actually use. If you have `className="bg-red-500"` in `components/Foo.tsx`, Tailwind sees it during the build and includes `bg-red-500` in the output CSS. If you don't reference a class anywhere, it's dropped.

This is the **purge / tree-shaking** that makes Tailwind's "thousands of utilities" not produce a 50MB CSS bundle. The output is usually under 30KB.

Note that we *don't* include `_source/**/*` in `content`. The `.jsx` reference file lives there, but we don't want its classes (which we may have changed) leaking into the build.

Step 5: `postcss.config.mjs`

```
const config = {
  plugins: { tailwindcss: {}, autoprefixer: {} },
};
export default config;
```

PostCSS is the CSS processor that runs Tailwind and adds vendor prefixes (e.g. `-webkit-` for older browsers). You'll never touch this file again after creating it.

Step 6: `.eslintrc.json`

```
{
  "extends": ["next/core-web-vitals"],
  "rules": {
    "react/no-unescaped-entities": "off"
  }
}
```

Two things:

- **`"extends": ["next/core-web-vitals"]`** — pulls in Next.js's recommended ESLint rules, which include "Core Web Vitals" performance hints (e.g. "use `next/image` instead of `<img>`").
- **`"react/no-unescaped-entities": "off"`** — disable the rule that complains about apostrophes (`don't`) and quotes inside JSX. This rule is more annoying than helpful in practice; we turn it off.

Step 7: `.prettierrc`

```
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "es5",
  "printWidth": 100
}
```

Prettier's job is to **end every code-style argument** in your team. With this config:

- Semicolons at the end of statements (`;`).
- Single quotes for strings (`'hello'` not `"hello"`).
- Trailing commas where ES5 allows (objects, arrays — not function args).
- Lines wrapped at 100 chars.

Pick a config, commit it, never argue about it again. **Prettier is a productivity multiplier specifically because it removes pointless decisions.**

Step 8: `.gitignore`

```
node_modules/
.next/
out/
.DS_Store
*.log
.env*.local
```

What we **don't** want in git:

- `node_modules/` — these are installed via `npm install`. Massive. ~400MB.
- `.next/` — Next.js's build cache. Regenerated on every build.
- `out/` — static export output, if you use `next export`.
- `.DS_Store` — macOS folder metadata. (You might not need this on Windows, but it's harmless.)
- `*.log` — log files.
- `.env*.local` — local secrets.

Never commit `node_modules`. It's the most common rookie mistake — and it bloats the repo to gigabytes.

Step 9: The `app/` directory (Next.js App Router)

Next.js 13+ uses **file-based routing**: every folder under `app/` becomes a URL route. The mapping is:

```
app/page.tsx       → /
app/about/page.tsx → /about
app/blog/[slug]/page.tsx → /blog/anything
```

We have only one route (/), so we need:

```
app/
  layout.tsx     ← wraps every page with the HTML shell
  page.tsx       ← the home page
  globals.css    ← global styles
```

app/layout.tsx

```
import type { Metadata } from 'next';
import { Roboto, Roboto_Slab } from 'next/font/google';
import './globals.css';

const roboto = Roboto({
  subsets: ['latin'],
  weight: ['300', '400', '500', '700', '900'],
  variable: '--font-roboto',
  display: 'swap',
});

const robotoSlab = Roboto_Slab({
  subsets: ['latin'],
  weight: ['600', '700', '900'],
  variable: '--font-roboto-slab',
  display: 'swap',
});

export const metadata: Metadata = {
  title: 'Snooker Fantasy League · Crucible 2026',
  description: 'Fantasy league standings for the 2026 World Snooker Champions
};

export default function RootLayout({ children }: { children: React.ReactNode
  return (
    <html lang="en" className={`${roboto.variable} ${robotoSlab.variable}`}>
      <body>{children}</body>
```

```
    </html>
  );
}
```

Three big concepts here:

1. This is a server component (no `'use client'` directive). Server components are the App Router's default. They render to HTML on the server (or at build time for static routes), ship zero JavaScript to the browser, and can `await` server-side data directly.

A server component **cannot use** `useState`, `useEffect`, event handlers like `onClick`, or any browser-only API. If you need any of those, you mark a file with `'use client'` at the top and React knows to ship it as JS.

This `layout.tsx` doesn't need any of those, so server component is correct.

```
const roboto = Roboto({ subsets: ['latin'], weight: [...], variable: '--font-
```

2. `next/font/google` for fonts This downloads the Roboto and Roboto Slab fonts at build time, hosts them on your own server, and exposes them as a CSS variable. **Your fonts never come from Google's servers in production.** That's a privacy + performance win:

- No request to `fonts.googleapis.com` (which can leak user data).
- The font is served from the same domain as your HTML, so there's one less DNS lookup.
- Next.js automatically applies `font-display: swap` and other best practices.

The CSS variable is set on `<html>` via `className={roboto.variable}` and you can use it from CSS (`font-family: var(--font-roboto)`).

```
export const metadata: Metadata = { title: '...', description: '...' };
```

3. The `metadata` export Next.js reads this and produces the corresponding `<title>` and `<meta name="description">` tags in the HTML head. **No need to manage `<head>` manually**, no need for `react-helmet`. Just export an object.

This is the “Metadata API” introduced in Next 13, and it’s a quiet superpower of the App Router. SEO improvements get easier the more you lean on it.

app/page.tsx

```
import SnookerFantasyLeague from '@/components/SnookerFantasyLeague';

export default function Page() {
  return <SnookerFantasyLeague />;
}
```

That’s the entire home page. It just renders the orchestrator component. Why?

Because `app/page.tsx` is a **route component** — it’s where Next.js looks when someone visits `/`. We want our actual application logic to live in `components/`, not under `app/`. The `app/page.tsx` is just a thin wrapper.

This is the **routing layer is separate from the app layer** pattern. Senior projects keep `app/` minimal and put logic in `components/`. Easier to test, easier to extract.

The `@/components/SnookerFantasyLeague` import uses our path alias from `tsconfig.json`. Without the alias, this would be `./../components/SnookerFantasyLeague` or worse.

app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --background: #fff8e7;
  --foreground: #1f2937;
}

html, body {
  padding: 0;
  margin: 0;
```

```
  background: var(--background);
  color: var(--foreground);
  font-family: var(--font-roboto), system-ui, sans-serif;
}

* {
  box-sizing: border-box;
}

button {
  font-family: inherit;
}
```

Three Tailwind directives plus some basic resets and a CSS variable theme. The most important line is `font-family: var(--font-roboto), ...` — that's where the font we set up in `layout.tsx` actually gets applied.

`box-sizing: border-box` on every element means **padding and border are included in width/height calculations**. Otherwise `width: 100px; padding: 10px` produces a 120px-wide element, which is maddening. Always include this.

Step 10: Install dependencies

```
npm install
```

This reads `package.json`, downloads all dependencies, and writes a `package-lock.json` that pins exact versions. First install on a fresh project takes a few minutes.

You'll see warnings about deprecated transitive dependencies — those are mostly fine for an app project. (Library authors care more.) The build will work.

Step 11: Run the dev server

```
npm run dev
```

Open `http://localhost:3000`. You should see — at this point — an error, because `SnookerFantasyLeague.tsx` doesn't exist yet. That's fine. **The error message is your**

friend — it confirms the routing is working, the build pipeline is running, and TypeScript is checking.

If instead you see a "page not found" or the dev server failed to start, scroll back through this lesson and verify:

- Every config file is named correctly (`.mjs` vs `.js` matters).
- `package.json` has all five dependencies.
- `app/layout.tsx` has the right structure.
- You're in the right folder.

What `create-next-app` would have done for you

You just hand-built what `create-next-app` does in 30 seconds. Was it worth it?

For learning, **yes**. You now know:

- What every config file is.
- Why `tsconfig.json` has `paths`, what `strict` does, what `noEmit` means.
- What `app/layout.tsx` is for.
- What server components are (the default!).
- Why the dev server starts where it does.

For a real project, **use `create-next-app`** and get back to building. But knowing what it did frees you to *modify* what it did. Most beginners are afraid to touch their `tsconfig.json` because they don't know what it does. You now do.

Vibe prompt you would have used

If you skipped the manual scaffold:

> *"Set up a new Next.js 14 project at the root of an empty folder. App Router, TypeScript strict mode, Tailwind CSS (set up but I'll mostly use inline styles), Prettier with single quotes / trailing commas, ESLint with the next/core-web-vitals preset. Add `lucide-react` and `recharts` as dependencies. The app has one route (/) that renders `<SnookerFantasyLeague />` from `@/components`. Configure `next/font/google` for Roboto and Roboto Slab and apply them via CSS variables. Don't generate any actual app components yet — just the scaffold."*

Specific, scoped, names every tool. Notice “Don’t generate any actual app components yet” — this is the same trick from Chapter 1 Lesson 1. Get the LLM to stop at scaffolding.

CHECK YOURSELF

1. **Why is `next` pinned to a specific version while `react` uses `^18.3.1`?** What’s the difference between pinned, caret (^), and tilde (~) version specifiers?
2. **You add a `console.log` to `app/page.tsx` and reload — where does the log appear, in your browser console or your terminal?** Why?
3. **A teammate adds `import { useState } from 'react'` to `app/layout.tsx` and gets an error. Why? What’s the fix?**
4. **`tsconfig.json` has `"paths": { "@/*": ["./*"] }`. What do you change to add a second alias `~/lib/*` mapping to `./lib/*`?**
5. **`reactStrictMode: true` causes some effects to run twice in dev. Why does this help catch bugs?** (Hint: it’s about side effects in render or in `useEffect` cleanup.)

When you’re ready, head to **03-creating-data-files.md** — the boring but essential lesson where we type out all the match data.